Qualifying the HTTP DSL

What a single `import Okapi` gets you, where names collide, and the worst case

Okapi

July 14, 2026

Introduction

Okapi's HTTP codec DSL is spread across roughly twenty modules — one core engine (`Okapi.Tree`), shared `Headers` and `Body` types, per-side `Request`/`Response` wrappers, and a full RFC 9651 Structured Field Values implementation nested underneath. Almost all of it funnels into one top-level `import Okapi`, which is enough for the overwhelming majority of real code. This document maps exactly where that stops being true: which names collide once you need something outside the curated top-level surface, why each collision exists, and what qualifier resolves it.

The rule the whole design follows: **`Okapi.HTTP.Request` and `Okapi.HTTP.Response` are each a complete, self-sufficient one-stop shop for their side.** A module that only builds requests, or only builds responses, imports the one it needs — unqualified — and gets everything: method/path/ query, header combinators, body combinators, the lot. A module that needs *both* sides at once necessarily needs to qualify, since the two sides export identical names by design (that's what makes each one complete on its own). Neither side gets a default, unqualified pass over the other — `Okapi` itself doesn't referee that choice for you.

Every claim here was checked against the real library, not asserted from memory — the worked example at the end is real, compiling code, verified against `lib/src` with GHCi (`cabal repl lib:okapi`, `:load`), not simplified past the point of actually type-checking.

Generic-deriving (`Path.derived`, `Query.derived`, `Okapi.HTTP.Headers.derived`, `GPath`/`GQuery`/`GHeaders`, `LitF`, `ConstF`) is out of scope for this pass — it has its own qualification story and will get its own document.

Part 1 — What `import Okapi` alone gives you

A single unqualified `import Okapi` brings in:

- **Request builders**: `req`, `reqGET`, `reqPOST`, `reqPUT`, `reqDELETE`, and the narrowing functions `method`, `path`, `query` — plus the `Request(..)` record itself.
- **Response builders**: `res`, `res200`, `res201`, `res204`, `res404`, `res500` — plus `Response(..)`. This is 6 of the ~47 status codes the library actually knows about.
- **Method singletons**: `GET`, `POST`, `PUT`, `DELETE` (4 of the 9 the library knows about), plus the `KnownMethod` type.
- **Status singletons**: `S200`, `S201`, `S204`, `S404`, `S500` (5 of ~47), plus `KnownStatus`.
- **Path**: `seg`, `seg_`, `lit`, `segs`.
- **Query**: `param`, `param'`, `param_`, `flag`, `flag'`, `list`, `list'`, `ArrayStyle(..)`.
- **Set-Cookie attributes**: `attr`, `attr'`, `secure`, `httpOnly` (4 of the 9 `Attributes` combinators).
- **Header combinators**: `field`, `field'`, `field_`, `contentType`, `fieldStruct`, `fieldBareItem`, `fieldItem`, `fieldList`, `fieldDict`, `cookie`, `cookie'`, `setCookie`, plus `MediaType(..)`. These are one shared definition each (free in a phantom request/response tag — see 2.1), so they're reachable unqualified regardless of which side, or both sides, a module builds.
- **Body combinators**: `json`, `jsonValue`, `form`, `noContent`, plus `None(..)` — same story as headers.

- **The generic `Tree` engine**: `SymTree`, `Leaf(..)`, `Info(..)`, `HasLeaf(..)` (which brings the `leaf` method itself into scope), `(=.)`, and the leaf combinators `int`, `int16`, `int32`, `int64`, `integer`, `bool`, `float`, `double`, `scientific`, `text`, `day`, `localTime`, `utcTime`, `timeOfDay`, `uuid`.
- **Contracts and responses**: `Contract(..)`, `(:->)`, `(:-<)`, `Cases`, `cases`, `getResponses`, `parseResponses`, `printResponses`.
- **The whole Endpoint/Handle/Client/Link/Morph layer**: `Endpoint(..)`, `endpoint`, `scope`, `route`, `catchAll`, `Handle(..)`, `handle`, `mount`, `run`, `endpoints`, `Transformer(..)`, `client`, `clientVia`, `fetch`, `clientFor`, `Link(..)`, `links`, `linksVia`, `morph`, `openApi`, `contractToOpenApi`. None of this has a collision story — every name here is already unique — so it's out of scope for the rest of this document.
- `IsoJson`.

Notably absent: the `headers`/`body` narrowing functions. Unlike the combinators above, `Request`'s `headers`/`body` and `Response`'s `headers`/ `body` are genuinely different functions — each updates a different concrete record type — so there's no single shared definition to give a free ride through `Okapi`. Reaching either always means `Req.headers`/ `Res.headers`/`Req.body`/`Res.body` — see Part 2.1 and 2.2.

That's the whole surface. Now, where it runs out.

Part 2 — The collision map

2.1 Header and body combinators are shared — free in a phantom side tag

`field`, `field'`, `field_`, `contentType`, `fieldStruct`, `fieldBareItem`, `fieldItem`, `fieldList`, `fieldDict`, `cookie`, `cookie'`, `setCookie` (headers) and `json`, `jsonValue`, `noContent`, `form` (body) are genuinely **one shared definition each** — not two competing concrete ones — reachable straight through bare `Okapi`, regardless of which side, or both sides, a given module builds. `Okapi.HTTP.Headers`/`Okapi.HTTP.Body` tag `Headers`/`Body` with a phantom `ForRequest`/`ForResponse` marker (`Okapi.HTTP.Side`, two empty data types, no `DataKinds` needed); the non-side-specific combinators are free in that phantom, so the exact same function works for either side — which one gets picked is resolved by ordinary type inference from however the result is eventually used (assigned to a `Request`'s `headers` field vs. a `Response`'s), the same mechanism that resolves `mempty` or `Nothing`. There's nothing left to collide on, so there's nothing left to qualify:

```
reviewReqHeaders = do
    field_ "x-service" "reviews"
    contentType JSON
    ...

reviewOkHeaders = do
    contentType JSON
    limit <- rp2 =. field "x-ratelimit-limit" int
    ...
```

`cookie`/`cookie'` (request-only), `setCookie` (response-only), and `form` (request-only body) are different in kind, but land in the same place: they're genuinely side-pinned — their GADT constructor's return type fixes the phantom outright (`Cookie :: ... -> Headers ForRequest a a`), so `setCookie` used where a request's headers are expected wouldn't type-check even if you tried — but pinned isn't the same thing as *colliding*. Three different, non-overlapping names never need a qualifier to disambiguate, so they ride along in `Okapi`'s unqualified export just like everything else here.

What genuinely still forces a qualifier is the `headers`/`body` *narrowing functions* that slot one of these codecs into an actual `Request`/ `Response` — see 2.2, the one real collision left in this whole area.

2.2 `headers`/`body` record-update is ambiguous with too few fields present

`Request` and `Response` share field names (`headers`, `body`), which is normally invisible — `DuplicateRecordFields`-style disambiguation resolves it as long as the update also touches a field unique to one side:

```
-- fine: `path`/`query` only exist on Request, so GHC infers the rest
req { path = reviewPath, query = reviewQuery, headers = ..., body = ... }
```

But an update touching *only* shared field names has nothing to disambiguate with:

```
-- ambiguous: `headers` and `body` both exist on Request and Response
res200 { headers = reviewOkHeaders, body = json @Review }
```

The fix is to use the narrowing *functions* instead of record-update syntax — they're not ambiguous, since each one has a concrete, already-resolved type. Unlike everything in 2.1, `headers`/`body` genuinely are two different functions per side (a real record update against a different concrete type, not just a different phantom tag), so neither is bundled into bare `Okapi` — this always means whichever of `Req.`/`Res.` matches the side you're narrowing:

```
reviewOk = Res.body (json @Review) (Res.headers reviewOkHeaders res200)
```

2.3 `path` collides with `URI`'s `path` field — in record-update syntax only

`Okapi.Mode.Link`'s `URI { path :: Text, query :: Text }` uses `NoFieldSelectors`, same as `Request`/`Response` — so it generates no top-level `path` function (bare `path` unqualified is *not* ambiguous as a plain function call, since `URI` contributes no such binding). But record-update syntax doesn't work that way: GHC's field-label resolution for `{ path = ... }` considers every `NoFieldSelectors` record with a `path` field project-wide, `URI` included, regardless of whether a real function of that name exists anywhere. So:

```
-- ambiguous: `path` is a field of both Request and URI
reqDELETE { path = deleteReviewPath }

-- fine: the narrowing *function* `path` has a concrete type already
path deleteReviewPath reqDELETE
```

The practical rule: once a record update's field set doesn't uniquely pin down one type, switch that one line to function-application style.

2.4 `param`/`flag`/`flag'`/`list` mean different things at different levels

Query-string parameters (`Okapi.HTTP.Request.Query`) and RFC 9651 item parameters (`Okapi.HTTP.Structured.Parameters`) are conceptually unrelated — a `?limit=5` query parameter and a `;window=60` Structured Field parameter — but they share combinator names almost exactly: `param`, `param'`, `param_`, `flag`, `flag'`. `Okapi.HTTP.Headers.Attributes` (Set-Cookie attributes) piles a *third* `flag`/`flag'` pair on top of that. All three are already unqualified-reachable or need to be:

```
import Okapi.HTTP.Structured.Parameters qualified as Params
import Okapi.HTTP.Headers.Attributes qualified as Attr
```

Query's `param`/`flag'` stay bare (that's what `Okapi` curates); Structured parameters go through `Params.param`/`Params.flag'`; Set-Cookie flags go through `Attr.flag`/`Attr.flag'`. In one function that touches all three — exactly the shape of the worked example below — every one of these needs its own qualifier to stay unambiguous.

2.5 `item`/`list`/`dict` exist at two levels of Structured Field Values

`Okapi.HTTP.Structured` exports `item`, `list`, `dict` as the three ways to wrap an `Item`/`List`/`Dictionary` codec into a full `Structured` value. `Okapi.HTTP.Structured.List` *also* exports `item` (a single List member, different type entirely — `Tree Item a a -> Tree List a a` vs. `Tree Item a a -> Tree Structured a a`). Both are genuinely useful in the same header definition — `fieldStruct`/`fieldList`/`fieldDict` on the `Headers` side already do the `Structured`-wrapping for you, so most code never touches `Struct.item`/`Struct.list`/`Struct.dict` directly, but the inner `Item`/`List`/`Dictionary`/`Parameters` modules are unavoidable the moment a header value has any internal structure at all (parameters, inner lists, multiple dictionary members):

```
import Okapi.HTTP.Structured.Item qualified as Item
import Okapi.HTTP.Structured.List qualified as SList
import Okapi.HTTP.Structured.Dictionary qualified as Dict
```

2.6 `Body` is unified with `Headers` — same phantom, same shape, plus two new pieces

`Okapi.HTTP.Request.Body` and `Okapi.HTTP.Response.Body` used to each define their own separate `Body` GADT with duplicated `Raw`/`Json`/`NoContent` logic (`Request`'s also had an extra `Form` case). They're now both `type` aliases — `RequestBody = Body ForRequest`, `ResponseBody = Body ForResponse` — over one shared `Okapi.HTTP.Body` core tagged with the same `Okapi.HTTP.Side` phantom `Headers` uses (2.1); `Form` is pinned to `Body ForRequest` right at its constructor, the same way `Cookie`/`SetCookie` are pinned on the `Headers` side. Two things came out of the original unification worth calling out on their own:

- `noContent`'s value type is a dedicated `None` (`data None = None`) instead of `()` — a no-body response reads as its own explicit thing rather than an incidental unit value.
- A new `jsonValue` combinator, decoding/encoding a body as a structured `Aeson.Value` directly — no `FromJSON`/`ToJSON`/`ToSchema` instances needed (`Aeson.Value` already has the JSON ones unconditionally), for callers who want dynamic JSON without defining a domain type for it.

`json`/`jsonValue`/`noContent`/`form` are reached exactly the way Part 2.1 describes for headers — bare, through `Okapi`, no qualifier needed on either side:

```
body = noContent          -- request side, IO None
body = json @Review       -- response side
```

`raw` for bodies is the one thing that *didn't* get folded into `Okapi` — see 2.9: `Headers` already has its own `raw`, and pulling both into the same unqualified surface would create a collision `Okapi` would then have to referee. It stays reachable only through `Okapi.HTTP.Request.Body`/`Okapi.HTTP.Response.Body` (or the core `Okapi.HTTP.Body`/`Okapi.HTTP.Headers` directly).

2.7 `MediaType` rides along with `contentType`

`MediaType` (the type `contentType`'s argument comes from — `JSON`, `HTML`, `PlainText`, …) is re-exported by `Okapi` alongside `contentType` itself (2.1), so `contentType JSON` needs nothing beyond the one unqualified import — no `Req.`/`Res.` prefix on either the function or the constructor.

2.8 `Data`, `Failure`, `Result`, and `Wai.Response` for anything past the DSL

The moment code steps past *defining* a contract into *consuming* one — rendering a `Cases` value with `printResponses`, parsing an incoming one with `parseResponses`, or just naming the decoded-value type — the record modules that hold decoded/error/raw-result shapes come into play, and none of them are reachable via `Okapi`:

```
import Okapi.Record.Data qualified as Data
import Network.Wai qualified as Wai
```

2.9 `raw`, `parser`, `printer`, `parseExact` are everywhere

Nearly every leaf module — `Path`, `Query`, the core `Headers`, `Structured`, `Item`, `List`, `Dictionary`, `Parameters`, `Attributes`, the core `Body` (and its `RequestBody`/`ResponseBody` wrappers) — exports its own `raw`, and most export `parser`/`printer` (`Okapi.Tree` itself also exports generic `parser`/`printer`). None of these are re-exported by `Okapi`, nor — for `Headers`/`Body` specifically — by the `Okapi.HTTP.Request`/`Okapi.HTTP.Response` facades either (2.6). This isn't usually a *conflict* to resolve so much as a standing fact: reaching for the escape-hatch pass-through (`raw`) or the low-level codec functions on any leaf module always means a qualified import, with no shortcut. See the reference table below for the full list.

Part 3 — The naming convention this all sits on

Two rules, applied consistently:

1. **Types and constructors stay fully spelled out.** `Request`, `Response`, `Headers`, `Structured`, `Attributes`, `Dictionary`, `Parameters`, `KnownMethod`, `KnownStatus` — never abbreviated.
2. **Term-level smart constructors abbreviate when the full word is long enough to matter**, using a commonly recognized short form: `request` → `req`, `response` → `res`, `segment`/`segment_`/`segments` → `seg`/`seg_`/`segs`, `attribute`/`attribute'` → `attr`/`attr'`, `dictionary` → `dict` (and `fieldDictionary` → `fieldDict` to match). Short names stay as they are (`field`, `param`, `flag`, `list`, `item`, `member`, `status`, `method`, `raw`) — there's nothing to gain by shortening a four-letter word.

One combinator, `bareItemEq`, broke rule 2 during this pass — every other "assert a fixed, known value" combinator in the DSL (`field_`, `param_` twice over, `seg_`) uses an underscore suffix on its "decode freely" sibling; `bareItemEq` sat right next to `bareItem` with a different naming style for the identical shape. It's now `bareItem_`.

The `Body` unification (2.6) added two more names under the same rules: `None` is a type, so it stays fully spelled rather than becoming something cryptic; `jsonValue` is a term-level combinator short enough already that there's nothing to abbreviate, matching `json`/`noContent`/`form` right next to it.

Part 4 — Reference: the rest

Functions that exist and are real, but don't naturally show up in a typical contract — the low-level codec plumbing (`parser`/`printer`/`parseExact` pairs, module-level `raw` pass-throughs), the method/status variants beyond what's curated, and a couple of specialized Structured Field Value leaf types.

Name	Module	Signature (abbreviated)	Purpose
`raw`	`Path`, `Query`, `Headers`, `Structured.Item`, `.List`, `.Dictionary`, `.Parameters`, `Headers.Attributes`, `Okapi.HTTP.Body` (and its `Request.Body`/`Response.Body` wrappers)	`Tree t ctx ctx`	Pass the whole context through unconstrained, per module.
`parser` / `printer`	same modules, plus `Okapi.Tree` itself	`Tree t i o -> Parser/Printer t _`	The low-level codec functions every `field`/`param`/etc. combinator is built from.

Name	Module	Signature (abbreviated)	Purpose
`parseExact`	`Path`, `Query`, `Headers`, `Structured`, `.Item`, `.List`, `.Dictionary`	`Tree t i o -> ctx -> Either (Either err leftover) o`	Require full consumption instead of tolerating leftover.
`reqPATCH`, `reqHEAD`, `reqOPTIONS`, `reqCONNECT`, `reqTRACE`	`Okapi.HTTP.Request`	`Request PATCH ...` etc.	The 5 method-fixed request builders beyond `reqGET`/`reqPOST`/`reqPUT`/`reqDELETE`.
`PATCH`, `HEAD`, `OPTIONS`, `CONNECT`, `TRACE`	`Okapi.HTTP.Request.Method`	`KnownMethod "..."`	The method singletons beyond `GET`/`POST`/`PUT`/`DELETE`.
`Method.method`, `Method.raw`, `Method.parse`, `Method.print`	`Okapi.HTTP.Request.Method`		Build/inspect a `Method` codec directly (`method`/`print` shadow `Prelude.print` if imported unqualified — the source hides `Prelude.print` itself for this reason).
`res100`…`res511` (minus the 6 curated)	`Okapi.HTTP.Response`	`Response (KnownStatus NNN) ...`	The rest of the ~47 pre-built response codecs.
`S100`…`S511` (minus the 5 curated), `SomeKnownStatus`, `allKnownStatuses`	`Okapi.HTTP.Response.Status`		The rest of the status singletons, plus the existential wrapper enumerating all of them.
`Status.status`, `Status.raw`, `Status.parse`, `Status.print`	`Okapi.HTTP.Response.Status`		Build/inspect a `Status` codec directly.
`Struct.item`, `Struct.list`, `Struct.dict` used standalone	`Okapi.HTTP.Structured`	`Tree Item/List/Dictionary a a -> Tree Structured a a`	The general escape hatch `fieldStruct` is sugar over — reach for this directly when a header value needs to switch between Item/List/Dictionary shapes generically.
`BareItem.displayString` / `DisplayString`	`Okapi.HTTP.Structured.BareItem`	`BareItem DisplayString`	RFC 9651 §3.3.8 Display String — percent-encoded, non-ASCII-safe text, distinct from the plain quoted `Text` (`sf-string`).
`BareItem.byteSequence` / `ByteSequence`	`Okapi.HTTP.Structured.BareItem`	`ByteSequence`	RFC 9651 §3.3.5 — base64, colon-delimited arbitrary bytes.

Name	Module	Signature (abbreviated)	Purpose
`BareItem.hasNonCanonicalInteger`	`Okapi.HTTP.Structured.Internal`	`ByteString -> Bool`	Flags RFC-legal-but-non-canonical integer syntax (leading zeros, `-0`) — used internally to exclude those shapes from round-trip properties.
`BareItem.renderInner` / `parseInnerToList`	`Okapi.HTTP.Structured.Internal`	`BareItem`	The primitives `List`'s inner-list handling is built from; only needed directly for hand-rolled inner-list logic outside `List`'s own combinators.
`SList.innerParser` / `innerPrinter`	`Okapi.HTTP.Structured.Internal`	`True InnerItems i o -> Parser/Printer InnerItems _`	Test/inspect an `InnerItems` value (the space-separated contents of one inner list) directly, the way `parser`/`printer` do for `Item`/`List` themselves.
`knownMethodToStd` / `extractMethod` / `knownStatusToHTTP` / `extractStatus`	`Method`, `Status`	—	Convert to/extract from the underlying `http-types` representations.

Part 5 — The worked example

A three-endpoint "product reviews" API, written to use as much of the DSL as naturally fits in one place while staying inside the scope from Part 1–4 (generic-deriving excluded, per the note above). It compiles as-is against this library — verified with `cabal repl lib:okapi` and `:load`.

```haskell
{-# LANGUAGE ApplicativeDo #-}
{-# LANGUAGE BlockArguments #-}
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE DeriveAnyClass #-}
{-# LANGUAGE DeriveGeneric #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TypeApplications #-}

module MaxQual where

import Data.Aeson qualified as Aeson
import Data.OpenApi (ToSchema)
import Data.Text (Text)
import Data.UUID (UUID)
import Data.ByteString (ByteString)
import Network.HTTP.Types qualified as Types
import Data.Maybe (fromMaybe)
import GHC.Generics (Generic)
```

```haskell
-- The one unqualified import: field/field'/field_/contentType/fieldStruct/
-- fieldBareItem/fieldItem/fieldList/fieldDict/cookie/cookie'/setCookie/
-- json/jsonValue/noContent/form/MediaType are all here too now -- they're
-- one shared definition each (free in a phantom ForRequest/ForResponse
-- tag), so there's no ambiguity left to force a qualifier on them, even in
-- a module using both sides at once like this one.
import Okapi

-- Only the *narrowing* functions -- `headers`, `body` -- still need
-- qualification: Request and Response really are different record types,
-- and record-update syntax can't disambiguate on field names alone (2.2).
import Okapi.HTTP.Response qualified as Res

-- Needed because Okapi only curates 4 of the 9 method singletons; anything
-- outside that curated set needs the defining module directly.
import Okapi.HTTP.Request.Method qualified as Method

-- Query: needed only for the ArrayStyle constructors Okapi doesn't
-- re-export (Exploded/CommaDelimited/SpaceDelimited/PipeDelimited).
import Okapi.HTTP.Request.Query qualified as Query

-- Set-Cookie attributes: Okapi only re-exports attr/attr'/secure/httpOnly.
import Okapi.HTTP.Headers.Attributes qualified as Attr

-- Structured Field Values (RFC 9651) -- none of this is reachable via
-- `Okapi` at all beyond the field-level fieldStruct/fieldBareItem/etc.
import Okapi.HTTP.Structured qualified as Struct
import Okapi.HTTP.Structured.BareItem qualified as BareItem
import Okapi.HTTP.Structured.Item qualified as Item
import Okapi.HTTP.Structured.List qualified as SList
import Okapi.HTTP.Structured.Dictionary qualified as Dict
import Okapi.HTTP.Structured.Parameters qualified as Params

-- For the parseResponses/printResponses demo -- neither of these record
-- modules is reachable via Okapi at all.
import Network.Wai qualified as Wai
import Okapi.Record.Data qualified as Data

-- ------------------------------------------------------------------------
-- Domain type
-- ------------------------------------------------------------------------

data Review = Review
    { reviewId :: Int
    , stars    :: Int
    , comment  :: Text
    }
    deriving (Generic, Show, Aeson.FromJSON, Aeson.ToJSON, ToSchema)

-- ------------------------------------------------------------------------
-- Endpoint 1: GET /v2/products/{productId}/reviews/{reviewId}
-- ------------------------------------------------------------------------
```

```haskell
clientToken :: BareItem.Token
clientToken = fromMaybe (error "bad literal token") (BareItem.mkToken "okapi-v2")

-- | A fixed marker token, plus a real build-number parameter riding along
--   with it -- same shape as the library's own `taggedParam` example:
--   'Item.bareItem_' composed with a value-producing sibling keeps the
--   whole thing's input free, no discard-projection needed.
clientItem = do
    Item.bareItem_ (BareItem.unToken clientToken)
    n <- Item.params (Params.param "build" (integer :: Leaf BareItem.BareItem Integer))
    pure n

reviewPath = do
    seg_ int 2
    lit "products"
    pid <- fst =. seg "productId" uuid
    lit "reviews"
    rid <- snd =. seg "reviewId" int
    pure (pid, rid)

reviewQuery = do
    verbose <- qp1 =. flag' "verbose"
    limit <- qp2 =. param' "limit" int
    tagFilter <- qp3 =. list' Query.Exploded "tag" text
    param_ "api" int 2
    fixedFmt <- qp4 =. param "format" text
    pure (verbose, limit, tagFilter, fixedFmt)
  where
    qp1 (a,_,_,_) = a
    qp2 (_,b,_,_) = b
    qp3 (_,_,c,_) = c
    qp4 (_,_,_,d) = d

-- Every combinator here is bare -- field_/contentType/fieldBareItem/
-- fieldItem/field'/fieldDict/cookie'/cookie are all unqualified via
-- `Okapi` now, the same as `seg`/`param`/etc. always were.
reviewReqHeaders = do
    field_ "x-service" "reviews"
    contentType JSON
    apiKey <- hp1 =. fieldBareItem "x-api-key" (BareItem.token :: Leaf BareItem.BareItem BareItem.Token)
    buildNum <- hp2 =. fieldItem "x-client" clientItem
    traceId <- hp3 =. field' "x-trace-id" uuid
    prefs <- hp4 =. fieldDict "prefer"
        ( (,) <$> (fst =. Dict.member "compact" (Item.bareItem (bool :: Leaf BareItem.BareItem Bool)))
              <*> (snd =. Dict.member' "notes" (Item.bareItem (bool :: Leaf BareItem.BareItem Bool)))
        )
    groups <- hp5 =. fieldDict "groups" (Dict.list "featured" (text :: Leaf BareItem.BareItem Text))
    sid <- hp6 =. cookie' "sid" uuid
    lang <- hp7 =. cookie "lang" text
    pure (apiKey, buildNum, traceId, prefs, groups, sid, lang)
  where
    hp1 (a,_,_,_,_,_,_) = a
    hp2 (_,b,_,_,_,_,_) = b
```

```
    hp3 (_,_,c,_,_,_,_) = c
    hp4 (_,_,_,d,_,_,_) = d
    hp5 (_,_,_,_,e,_,_) = e
    hp6 (_,_,_,_,_,f,_) = f
    hp7 (_,_,_,_,_,_,g) = g

getReview =
    req { path = reviewPath
        , query = reviewQuery
        , headers = reviewReqHeaders
        , body = noContent
        }

-- ------------------------------------------------------------------------------
-- Endpoint 1's responses
-- ------------------------------------------------------------------------------

sessionAttrs = do
    age <- ap1 =. Attr.maxAge
    dom <- ap2 =. Attr.domain
    pth <- ap3 =. Attr.path
    ap4 =. secure
    ap5 =. httpOnly
    sameSite <- ap6 =. attr "SameSite" (leaf :: Leaf Attr.Attributes ByteString)
    priority <- ap7 =. attr' "Priority" (leaf :: Leaf Attr.Attributes ByteString)
    ap8 =. Attr.flag "Partitioned"
    debug <- ap9 =. Attr.flag' "Debug"
    pure (age, dom, pth, sameSite, priority, debug)
  where
    ap1 (a,_,_,_,_,_) = a
    ap2 (_,b,_,_,_,_) = b
    ap3 (_,_,c,_,_,_) = c
    ap4 _ = ()
    ap5 _ = ()
    ap6 (_,_,_,d,_,_) = d
    ap7 (_,_,_,_,e,_) = e
    ap8 _ = ()
    ap9 (_,_,_,_,_,f) = f

cacheTags = SList.items (Item.bareItem (text :: Leaf BareItem.BareItem Text))

relatedIds =
    (,) <$> (fst =. SList.item (Item.bareItem (integer :: Leaf BareItem.BareItem Integer)))
        <*> (snd =. SList.item (Item.bareItem (integer :: Leaf BareItem.BareItem Integer)))

scoreBuckets =
    (,) <$> (fst =. SList.innerList (integer :: Leaf BareItem.BareItem Integer))
        <*> (snd =. SList.innerList (integer :: Leaf BareItem.BareItem Integer))

batchGroups =
    (,) <$> (fst =. SList.innerListOf
                (SList.innerItem (Item.bareItem (text :: Leaf BareItem.BareItem Text)))
                (Params.param "lvl" (integer :: Leaf BareItem.BareItem Integer)))
```

```
        <*> (snd =. SList.innerListOf
                (SList.innerItems (Item.bareItem (integer :: Leaf BareItem.BareItem Integer)))
                (Params.param' "lvl" (integer :: Leaf BareItem.BareItem Integer)))

-- Same story on the response side -- `contentType`/`field`/`fieldItem`/
-- `fieldList`/`setCookie` are the exact same bare names used unqualified
-- above for the request; using both sides in the same module doesn't
-- force a qualifier on any of these any more, only on `Res.body`/
-- `Res.headers` below (2.1, 2.2).
reviewOkHeaders = do
    rp1 =. contentType JSON
    limit <- rp2 =. field "x-ratelimit-limit" int
    rateItem <- rp3 =. fieldItem "x-ratelimit"
        ( (,) <$> (fst =. Item.bareItem (integer :: Leaf BareItem.BareItem Integer))
              <*> (snd =. Item.params
                    ( (,) <$> (fst =. Params.param "window" (integer :: Leaf BareItem.BareItem Integer))
                          <*> (snd =. Params.flag' "burst")
                    ))
        )
    tags <- rp4 =. fieldList "cache-tags" cacheTags
    related <- rp5 =. fieldList "x-related-ids" relatedIds
    buckets <- rp6 =. fieldList "x-score-buckets" scoreBuckets
    groups <- rp7 =. fieldList "x-batch-groups" batchGroups
    session <- rp8 =. setCookie "session" uuid sessionAttrs
    pure (limit, rateItem, tags, related, buckets, groups, session)
  where
    rp1 _ = ()
    rp2 (a,_,_,_,_,_,_) = a
    rp3 (_,b,_,_,_,_,_) = b
    rp4 (_,_,c,_,_,_,_) = c
    rp5 (_,_,_,d,_,_,_) = d
    rp6 (_,_,_,_,e,_,_) = e
    rp7 (_,_,_,_,_,f,_) = f
    rp8 (_,_,_,_,_,_,g) = g

-- `headers`/`body` are ambiguous as a record update here (both `Request`
-- and `Response` have fields by those names, and nothing else in this
-- particular update disambiguates which record is meant) -- the narrowing
-- *functions* sidestep that, but they're the one thing that still needs
-- `Res.` (2.2) -- `json` itself is bare, same as everything else above.
reviewOk = Res.body (json @Review) (Res.headers reviewOkHeaders res200)

getReviewContract = getReview :-> reviewOk

-- ---------------------------------------------------------------------------
-- Endpoint 2: GET /products/{productId}/reviews/tags/{tag}+  -- `segs`
-- ---------------------------------------------------------------------------

listPath = do
    lit "products"
    pid <- fst =. seg "productId" uuid
    lit "reviews"
    lit "tags"
```

```
    tags <- snd =. segs text
    pure (pid, tags)

listReviews = req
    { method = Method.method GET
    , path = listPath
    , body = noContent
    }

listOk = Res.body (json @[Review]) res200

listReviewsContract = listReviews :-> listOk

-- ---------------------------------------------------------------------------
-- Endpoint 3: a `Cases` contract with two response alternatives -- `Cases`,
-- `cases`, `getResponses`, `parseResponses`, `printResponses`
-- ---------------------------------------------------------------------------

deleteReviewPath = do
    lit "products"
    pid <- fst =. seg "productId" uuid
    lit "reviews"
    rid <- snd =. seg "reviewId" int
    pure (pid, rid)

-- | `reqDELETE { path = deleteReviewPath }` is ambiguous even though `URI`
--   uses NoFieldSelectors (no real top-level `path` function comes from
--   it) -- the record-*update* syntax itself considers every NoFieldSelectors
--   record with a `path` field, `URI` included, regardless of whether a
--   plain function of that name exists. The narrowing *function* `path`
--   has no such ambiguity, so function-application style sidesteps it.
deleteReviewReq = path deleteReviewPath reqDELETE

deletedRes = Res.body noContent res204
notFoundRes = Res.body (json @Text) res404

data DeleteReviewResponses f
    = Deleted (f S204 Types.ResponseHeaders (IO None))
    | ReviewNotFound (f S404 Types.ResponseHeaders (IO Text))
    deriving (Generic, Cases)

deleteReviewCases = cases @DeleteReviewResponses deletedRes notFoundRes

deleteReviewContract = deleteReviewReq :-< deleteReviewCases

-- | How many branches this `Cases` value actually has right now.
deleteReviewBranchCount :: Int
deleteReviewBranchCount = length (getResponses deleteReviewCases)

-- | Render the `Deleted` branch straight to a real 'Wai.Response' -- and
--   parse an incoming one back into whichever branch it actually matches.
--   Neither `Okapi.Record.Data` nor `Network.Wai` is reachable via `Okapi`
--   at all.
```

```haskell
demoPrintDeleted :: IO Wai.Response
demoPrintDeleted = printResponses deleteReviewCases (Deleted (Data.Response S204 [] (pure None)))

demoParseIncoming waiResponse = parseResponses deleteReviewCases waiResponse
```

What this example actually demonstrates

- **Endpoint 1** (`getReviewContract`) is the dense one: a path mixing a literal-integer version segment (`seg_`), literal text segments (`lit`), and typed segments (`seg`); a query using every combinator (`param`/`param'`/`param_`/`flag'`/`list'`); request headers spanning a plain fixed-value assertion (`field_`), a `Structured.BareItem` token field, a composed `Item` (fixed marker token *plus* a real parameter, demonstrating why `bareItem_` needs a value-producing sibling to stay useful), a `Dictionary` with both required and optional members, a second `Dictionary` whose member is itself an inner list, and both request cookies — every single one of those combinators bare, no `Req.`/`Res.` prefix anywhere (2.1). The response side reuses the exact same bare names (`contentType`, `field`, `fieldItem`, `fieldList`, `setCookie`), builds a parameterized `Item` header from scratch, four different `List`-shaped headers (whole-list, chained single items, chained inner lists, and the full `innerListOf`/`innerItem`/`innerItems` heterogeneous-composition RFC 9651 §3.1.1 shape), and a `Set-Cookie` combining four pre-built `Attributes` combinators, two custom ones via bare `attr`/`attr'`, and two more via qualified `Attr.flag`/`Attr.flag'`. The *only* qualifier either side of this endpoint actually needs is `Res.` on `body`/`headers` themselves (2.2) — genuinely different functions per side, unlike everything else here.
- **Endpoint 2** (`listReviewsContract`) exists mainly for `segs` (a trailing non-empty catch-all of tag segments) and to show `Method.method` used directly instead of one of the pre-built `reqGET` starting points.
- **Endpoint 3** (`deleteReviewContract`) is the `Cases` demo: two response alternatives, plus `getResponses`, and both directions of `printResponses`/`parseResponses` against a real `Wai.Response`.

Across the whole thing, only eight qualifiers are load-bearing: `Res` (for `body`/`headers` alone — nothing else needs it), `Method`, `Query`, `Attr`, `BareItem`, `Item`, `SList`, `Dict`, `Params`, `Wai`, `Data` — plus `Struct`, imported but never actually typed in the body, since `fieldDict`/ `fieldItem`/`fieldList` cover its job (Part 4). Compare that to the ten-plus qualifiers this same example needed before headers and bodies were shared across sides (`Res`, `ResH`, `ReqBody`, `ResBody`, …): nearly everything that used to force a `Req.`/`Res.` prefix — every header field, every body combinator, `MediaType` itself — doesn't any more. What's left is the genuinely structural stuff: Structured Field Values nested three levels deep, Set-Cookie attributes, and the two narrowing functions that really do update different concrete records.